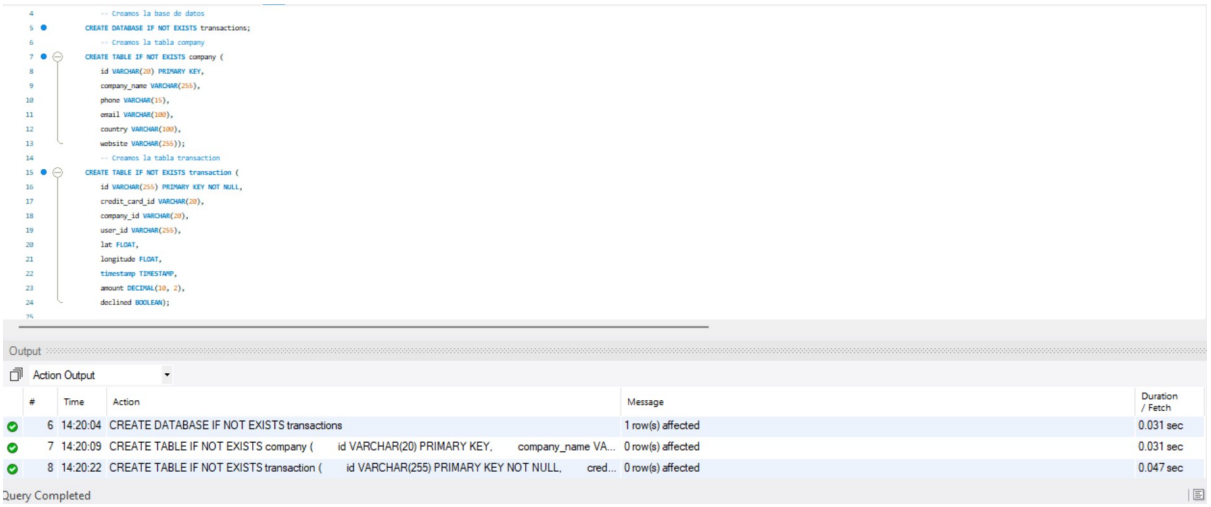
Nombre:	Sara Faiza Boul Hanna
P2P:	Carlos
Fecha:	12/11/2026
GitHub:	*https://github.com/SFBcode/data-analysis-portfolio/tree/main/Cybernarium/SQL-Projects/Sprint%201*

Nivel 1

Ex.1 Database Schema

Based on the attached documents (data_structure and input_data), import the two tables. Show the main features of the schema created and explain the different tables and variables that exist. Be sure to include a diagram that illustrates the relationship between the different tables and variables.

Execution (Workbench):



```
-- Creamos la base de datos
CREATE DATABASE IF NOT EXISTS transactions;
-- Creamos la tabla company
CREATE TABLE IF NOT EXISTS company (
    id VARCHAR(20) PRIMARY KEY,
    company_name VARCHAR(255),
    phone VARCHAR(15),
    email VARCHAR(100),
    country VARCHAR(100),
    website VARCHAR(255));
    -- Creamos la tabla transaction
CREATE TABLE IF NOT EXISTS transaction (
    id VARCHAR(255) PRIMARY KEY NOT NULL,
    credit_card_id VARCHAR(20),
    company_id VARCHAR(20),
    user_id VARCHAR(255),
    lat FLOAT,
    longitude FLOAT,
    timestamp TIMESTAMP,
    amount DECIMAL(10, 2),
    declined BOOLEAN);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
6	14:20:04	CREATE DATABASE IF NOT EXISTS transactions	1 row(s) affected	0.031 sec
7	14:20:09	CREATE TABLE IF NOT EXISTS company (id VARCHAR(20) PRIMARY KEY, company_name VA...	0 row(s) affected	0.031 sec
8	14:20:22	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY NOT NULL, cred...	0 row(s) affected	0.047 sec

Query Completed



```
USE transactions;

-- Insertamos datos de company
INSERT INTO company (id, company_name, phone, email, country, website) VALUES (  'b-2222', 'Ac Fermentum Incorporated', '06 85 56 52 33', 'donec.porttitor.tellus@yahoo.net', 'Germany', 'https
INSERT INTO company (id, company_name, phone, email, country, website) VALUES (  'b-2226', 'Magna A Neque Industries', '04 14 44 64 62', 'risus.donec.nibh@icloud.org', 'Australia', 'https://w
INSERT INTO company (id, company_name, phone, email, country, website) VALUES (  'b-2230', 'Fusce Corp.', '08 14 97 58 85', 'risus@protonmail.edu', 'United States', 'https://pinterest.com/sub
INSERT INTO company (id, company_name, phone, email, country, website) VALUES (  'b-2234', 'Convallis In Incorporated', '06 66 57 29 50', 'mauris.ut@aol.couk', 'Germany', 'https://cnn.com/use
INSERT INTO company (id, company_name, phone, email, country, website) VALUES (  'b-2238', 'Ante Iaculis Nec Foundation', '08 23 04 99 53', 'sed.dictum.proin@outlook.ca', 'New Zealand', 'http
INSERT INTO company (id, company_name, phone, email, country, website) VALUES (  'b-2242', 'Donec Ltd', '01 25 51 37 37', 'at.iaculis@hotmail.couk', 'Norway', 'https://nytimes.com/user/110');
INSERT INTO company (id, company_name, phone, email, country, website) VALUES (  'b-2246', 'Sed Nunc Ltd', '02 62 64 73 48', 'nibh@yahoo.org', 'United Kingdom', 'https://cnn.com/one');
INSERT INTO company (id, company_name, phone, email, country, website) VALUES (  'b-2250', 'Amet Nulla Donec Corporation', '07 15 25 14 74', 'mattis.integer.eu@protonmail.net', 'Italy', 'http
INSERT INTO company (id, company_name, phone, email, country, website) VALUES (  'b-2254', 'Nascetur Ridiculus Mus Inc.', '06 26 87 61 84', 'suspendisse.dui@icloud.net', 'United States', 'htt
INSERT INTO company (id, company_name, phone, email, country, website) VALUES (  'b-2258', 'Vestibulum Lorem PC', '02 02 87 33 40', 'aenean.massa.integer@aol.net', 'Belgium', 'https://pintere
INSERT INTO company (id, company_name, phone, email, country, website) VALUES (  'b-2262', 'Gravida Sagittis LLP', '03 81 28 33 97', 'turpis.vitae@google.ca', 'Sweden', 'https://naver.com/sit
INSERT INTO company (id, company_name, phone, email, country, website) VALUES (  'b-2266', 'Mus Aenean Eget Foundation', '06 25 15 52 43', 'mi.duis@hotmail.net', 'Sweden', 'https://instagram.
INSERT INTO company (id, company_name, phone, email, country, website) VALUES (  'b-2270', 'Dis Parturient Institute', '05 36 29 78 74', 'purus@protonmail.org', 'Ireland', 'https://google.com
INSERT INTO company (id, company_name, phone, email, country, website) VALUES (  'b-2274', 'Sed LLC', '01 63 16 26 52', 'at@outlook.com', 'Belgium', 'https://reddit.com/fr');
INSERT INTO company (id, company_name, phone, email, country, website) VALUES (  'b-2278', 'Arcu LLP', '06 46 04 41 45', 'dui@aol.ca', 'Norway', 'https://yahoo.com/sub');
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
100103	14:41:10	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, dec...	1 row(s) affected	0.000 sec
100104	14:41:10	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, dec...	1 row(s) affected	0.015 sec

```
38
39 •    ALTER TABLE transaction
40     ADD CONSTRAINT fk_company
41     FOREIGN KEY (company_id) REFERENCES company(id)
42     ON DELETE SET NULL;
43
```

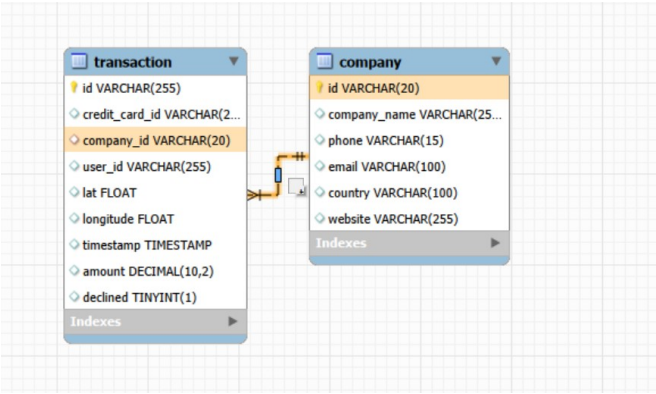
Output

Action Output

#	Time	Action	Message	Duration / Fetch
12	14:33:08	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY N...	0 row(s) affected	0.047 sec
13	14:50:32	ALTER TABLE transaction ADD CONSTRAINT fk_company FOREIGN KEY (comp...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	2.265 sec

- I am creating a MySQL database called “transactions” to store company information and financial transactions.
- I am defining tables, their attributes, primary keys, and foreign keys to establish relationships between “company” and “transaction”.

Diagram (Database -> Reverse Engineer):



Ex.2 Queries

*Using it JOIN you will make the following queries: * List of countries that are generating sales. * From how many countries are sales generated? * Identify the company with the highest average sales.*

```
32         -- -- Ex.2
33
34         -- List of countries that are generating sales.
35 •    SELECT DISTINCT c.country
36     FROM company AS c
37     INNER JOIN `transaction` AS t
38         ON c.id = t.company_id
39     WHERE t.declined = 0;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

country
Netherlands
Sweden
Ireland
United States
Belgium
Canada
Germany

Result 2 | Read Or

Output

Action Output

#	Time	Action	Message	Duration / Fetch
14	14:59:53	SELECT DISTINCT c.country FROM company AS c INNER JOIN `transaction` AS t ...	15 row(s) returned	0.531 sec / 0.000 sec
15	15:00:05	SELECT DISTINCT c.country FROM company AS c INNER JOIN `transaction` AS t ...	15 row(s) returned	0.547 sec / 0.000 sec

```sql
-- From how many countries are sales generated?
SELECT COUNT(DISTINCT c.country) AS total_countries
FROM company AS c
INNER JOIN `transaction` AS t
    ON c.id = t.company_id
WHERE t.declined = 0;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_countries
15

Result 3

Output

Action Output

#	Time	Action	Message	Duration / Fetch
15	15:00:05	SELECT DISTINCT c.country FROM company AS c INNER JOIN `transaction` AS t ...	15 row(s) returned	0.547 sec / 0.000 sec
16	15:07:21	SELECT COUNT(DISTINCT c.country) AS total_countries FROM company AS c IN...	1 row(s) returned	0.640 sec / 0.000 sec

- We got total of 15 distinct countries

```sql
-- Identify the company with the highest average sales.
SELECT c.id, c.company_name, ROUND(AVG(t.amount), 2) AS average_sales
FROM company AS c
INNER JOIN `transaction` AS t ON c.id = t.company_id
WHERE t.declined = 0
GROUP BY c.id, c.company_name
ORDER BY average_sales DESC
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	company_name	average_sales
b-2222	Ac Fermentum Incorporated	284.91

Result 4

Output

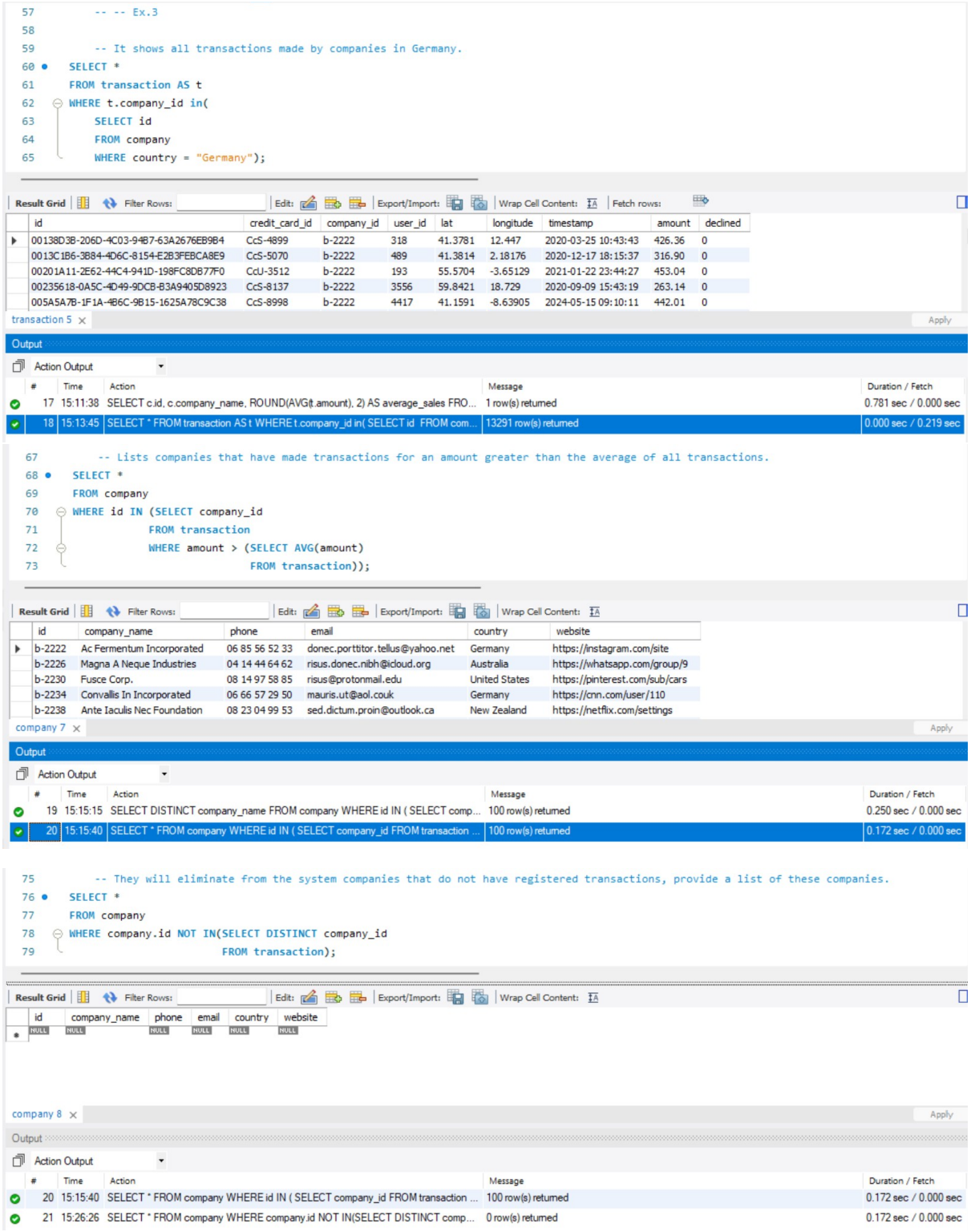
Action Output

#	Time	Action	Message	Duration / Fetch
16	15:07:21	SELECT COUNT(DISTINCT c.country) AS total_countries FROM company AS c IN...	1 row(s) returned	0.640 sec / 0.000 sec
17	15:11:38	SELECT c.id, c.company_name, ROUND(AVG(t.amount), 2) AS average_sales FRO...	1 row(s) returned	0.781 sec / 0.000 sec

- The company's highest average sale is 284.91

Ex.3 Subqueries

*Using only subqueries (without using JOIN): * It shows all transactions made by companies in Germany. * Lists companies that have made transactions for an amount greater than the average of all transactions. * They will eliminate from the system companies that do not have registered transactions, provide a list of these companies.*



```
-- -- Ex.3

-- It shows all transactions made by companies in Germany.
SELECT *
FROM transaction AS t
WHERE t.company_id in(
    SELECT id
    FROM company
    WHERE country = "Germany");
```

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00138D3B-206D-4C03-94B7-63A2676EB9B4	CcS-4899	b-2222	318	41.3781	12.447	2020-03-25 10:43:43	426.36	0
0013C1B6-3B84-4D6C-8154-E2B3FEBCA8E9	CcS-5070	b-2222	489	41.3814	2.18176	2020-12-17 18:15:37	316.90	0
00201A11-2E62-44C4-941D-198FC8DB77F0	CcU-3512	b-2222	193	55.5704	-3.65129	2021-01-22 23:44:27	453.04	0
00235618-0A5C-4D49-9DCB-B3A9405D8923	CcS-8137	b-2222	3556	59.8421	18.729	2020-09-09 15:43:19	263.14	0
005A5A7B-1F1A-4B6C-9B15-1625A78C9C38	CcS-8998	b-2222	4417	41.1591	-8.63905	2024-05-15 09:10:11	442.01	0

transaction 5

Output

Action Output

#	Time	Action	Message	Duration / Fetch
17	15:11:38	SELECT c.id, c.company_name, ROUND(AVG(t.amount), 2) AS average_sales FRO...	1 row(s) returned	0.781 sec / 0.000 sec
18	15:13:45	SELECT * FROM transaction AS t WHERE t.company_id in(SELECT id FROM com...	13291 row(s) returned	0.000 sec / 0.219 sec

```
-- Lists companies that have made transactions for an amount greater than the average of all transactions.
SELECT *
FROM company
WHERE id IN (SELECT company_id
             FROM transaction
             WHERE amount > (SELECT AVG(amount)
                             FROM transaction));
```

id	company_name	phone	email	country	website
b-2222	Ac Fermentum Incorporated	06 85 56 52 33	donec.porttitor.tellus@yahoo.net	Germany	https://instagram.com/site
b-2226	Magna A Neque Industries	04 14 44 64 62	risus.donec.nibh@icloud.org	Australia	https://whatsapp.com/group/9
b-2230	Fusce Corp.	08 14 97 58 85	risus@protonmail.edu	United States	https://pinterest.com/sub/cars
b-2234	Convallis In Incorporated	06 66 57 29 50	mauris.ut@aol.couk	Germany	https://cnn.com/user/110
b-2238	Ante Iaculis Nec Foundation	08 23 04 99 53	sed.dictum.proin@outlook.ca	New Zealand	https://netflix.com/settings

company 7

Output

Action Output

#	Time	Action	Message	Duration / Fetch
19	15:15:15	SELECT DISTINCT company_name FROM company WHERE id IN (SELECT comp...	100 row(s) returned	0.250 sec / 0.000 sec
20	15:15:40	SELECT * FROM company WHERE id IN (SELECT company_id FROM transaction ...	100 row(s) returned	0.172 sec / 0.000 sec

```
-- They will eliminate from the system companies that do not have registered transactions, provide a list of these companies.
SELECT *
FROM company
WHERE company.id NOT IN(SELECT DISTINCT company_id
                        FROM transaction);
```

id	company_name	phone	email	country	website
NULL	NULL	NULL	NULL	NULL	NULL

company 8

Output

Action Output

#	Time	Action	Message	Duration / Fetch
20	15:15:40	SELECT * FROM company WHERE id IN (SELECT company_id FROM transaction ...	100 row(s) returned	0.172 sec / 0.000 sec
21	15:26:26	SELECT * FROM company WHERE company.id NOT IN(SELECT DISTINCT comp...	0 row(s) returned	0.172 sec / 0.000 sec

- There are 100 companies in total. All of them appear in the second query because each company has at least one transaction above the average value.
- All companies in the database have made at least one transaction. That is why the third query returns no results.

Ex.4 Create table credit_card

Your task is to design and create a table called "credit_card" that stores crucial details about credit cards. The new table must be able to uniquely identify each card and establish an appropriate relationship with the other two tables ("transaction" and "company"). After creating the table, you will need to enter the information in the document called "credit_input_data". Remember to show the diagram and provide a brief description of it.

```sql
CREATE TABLE transactions.credit_card (
    id VARCHAR(20) PRIMARY KEY NOT NULL,
    iban VARCHAR(255),
    pan VARCHAR(255),
    pin VARCHAR(255),
    cvv VARCHAR(255),
    expiring_date VARCHAR(255)
);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
21	15:26:26	SELECT * FROM company WHERE company.id NOT IN(SELECT DISTINCT comp...	0 row(s) returned	0.172 sec / 0.000 sec
22	15:40:59	CREATE TABLE transactions.credit_card (id VARCHAR(20) PRIMARY KEY NOT N...	0 row(s) affected	0.079 sec

```sql
USE transactions;

-- Insertamos datos de credit_card
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2938', 'TR301950312213576817638661', '5424465566813633', '3257'
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2945', 'DO28054763748537475216568689', '5142423821948828', '908
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2952', 'BG45IVQL52710525608255', '4556 453 55 5287', '4598', '4
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2959', 'CR7242477244335841535', '372461377349375', '3583', '667
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2966', 'BG72LKTQ15627628377363', '448566 886747 7265', '4900',
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2973', 'PT87806228135092429456346', '544 58654 54343 384', '876
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

id	iban	pan	pin	cvv	expiring_date
CcS-4857	XX4857591835292505850771	2314242385113924	1819	467	09/27/25
CcS-4858	XX8581768137002436094025	6582720299715533	3964	817	12/28/28
CcS-4859	XX78269309491423553609370	8861684536289642	4983	277	11/26/26
CcS-4860	XX55595903688353046452 99	2481155515498459	6876	661	07/27/27
CcS-4861	XX20351828771951916273 07	1308930301149557	5710	398	04/25/26

credit_card 1 | Apply | Revert

Output

Action Output

#	Time	Action	Message	Duration / Fetch
5002	15:44:03	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-958...	1 row(s) affected	0.015 sec
5003	15:44:42	select * from credit_card	5000 row(s) returned	0.000 sec / 0.015 sec

```sql
UPDATE credit_card
SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y')
LIMIT 5000;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
22	15:40:59	CREATE TABLE transactions.credit_card (id VARCHAR(20) PRIMARY KEY NOT N...	0 row(s) affected	0.079 sec
23	16:11:55	UPDATE credit_card SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/...	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0	0.266 sec

```sql
ALTER TABLE credit_card
MODIFY expiring_date DATE;

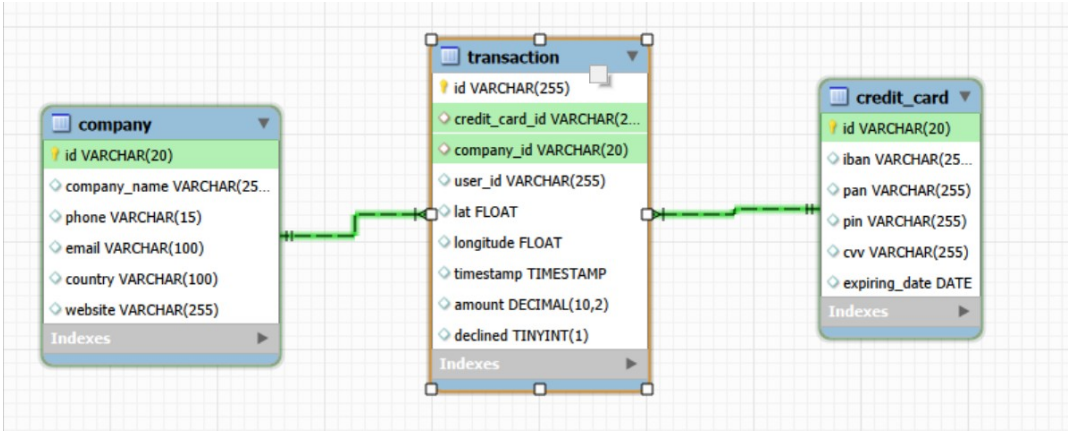
ALTER TABLE transaction
ADD CONSTRAINT fk_credit_card
FOREIGN KEY (credit_card_id)
REFERENCES credit_card(id)
ON DELETE SET NULL;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
24	16:17:13	ALTER TABLE credit_card MODIFY expiring_date DATE	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.234 sec
25	16:17:17	ALTER TABLE transaction ADD CONSTRAINT fk_credit_card FOREIGN KEY (cred...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	3.922 sec

Diagram:



Ex.5

The Human Resources department has identified an error in the account number associated with the credit card with ID CcU-2938. The information that should be displayed for this record is: TR323456312213576817699999. Remember to show that the change was made.

```
104             -- -- Ex.5
105         -- find the information for id CcU-2938:
106 •   SELECT *
107     FROM credit_card
108     WHERE id = "CcU-2938";
109
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR301950312213576817638...	5424465566813633	3257	984	2022-10-30
NULL	NULL	NULL	NULL	NULL	NULL

credit_card 9 | Apply | Revert

Output

Action Output

#	Time	Action	Message	Duration / Fetch
25	16:17:17	ALTER TABLE transaction ADD CONSTRAINT fk_credit_card FOREIGN KEY (cred...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	3.922 sec
26	16:25:22	SELECT * FROM credit_card WHERE id = "CcU-2938"	1 row(s) returned	0.000 sec / 0.000 sec

```
110         -- update the iban number:
111 •   UPDATE credit_card
112     SET iban = "TR323456312213576817699999"
113     WHERE id = "CcU-2938";
114
115         -- find the information for id CcU-2938:
116 •   SELECT *
117     FROM credit_card
118     WHERE id = "CcU-2938";
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	2022-10-30
NULL	NULL	NULL	NULL	NULL	NULL

credit_card 10 | Apply | Revert

Output

Action Output

#	Time	Action	Message	Duration / Fetch
27	16:30:16	UPDATE credit_card SET iban = "TR323456312213576817699999" WHERE id = "...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.032 sec
28	16:30:31	SELECT * FROM credit_card WHERE id = "CcU-2938"	1 row(s) returned	0.000 sec / 0.000 sec

Ex.6

In the "transaction" table, enter a new transaction with the following information:

Id 108B1D1D-5B23-A76C-55EF-C568E49A99DD credit_card_id CcU-9999 company_id b-9999 user_id 9999 lat 829.999 longitude -117.999 amount 111.11 declined 0

```
123             -- -- Ex.6
124
125         -- Add id to credit_card and id to company
126 •   INSERT INTO transactions.credit_card (id)
127     VALUES ('CcU-9999');
128
129 •   INSERT INTO transactions.company (id)
130     VALUES ('b-9999');
131
132         -- insert info to transaction:
133 •   INSERT INTO transaction
134     (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
135     VALUES
136     ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', 9999, 829.999, -117.999, 111.11, 0);
137
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
29	16:38:16	INSERT INTO transactions.credit_card (id) VALUES ('CcU-9999')	1 row(s) affected	0.063 sec
30	16:38:21	INSERT INTO transactions.company (id) VALUES ('b-9999')	1 row(s) affected	0.047 sec
31	16:38:25	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, am...	1 row(s) affected	0.047 sec

Ex.7

Human resources has asked you to delete the "pan" column from the credit_card table. Remember to show the change made.

```
138             -- -- Ex.7
139
140 •   DESCRIBE credit_card;
141
142 •   ALTER TABLE credit_card
143     DROP COLUMN pan;
144
145 •   DESCRIBE credit_card;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id	varchar(20)	NO	PRI	NULL	
iban	varchar(255)	YES		NULL	
pin	varchar(255)	YES		NULL	
cvv	varchar(255)	YES		NULL	
expiring_date	date	YES		NULL	

Result 13 ×

Output

Action Output

#	Time	Action	Message	Duration / Fetch
33	17:12:19	DESCRIBE credit_card	6 row(s) returned	0.015 sec / 0.000 sec
34	17:12:29	ALTER TABLE credit_card DROP COLUMN pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.094 sec
35	17:12:33	DESCRIBE credit_card	5 row(s) returned	0.000 sec / 0.000 sec

Ex.8

*Download the CSV files that **you will find in the resources section** :*

- *american_users.csv*
- *european_users.csv*
- *companies.csv*
- *credit_cards.csv*
- *transactions.csv*

Study them and design a database with a star schema that contains at least 4 tables from which you can perform the following queries:
We will use the products.csv table later.

```sql
-- --Ex.8

CREATE DATABASE IF NOT EXISTS sales;
USE sales;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
36	17:17:09	CREATE DATABASE IF NOT EXISTS sales	1 row(s) affected	0.031 sec
37	17:17:15	USE sales	0 row(s) affected	0.000 sec

```sql
    -- Dimention table users:
CREATE TABLE IF NOT EXISTS users (
id VARCHAR(255) NULL,
name VARCHAR(255) NULL,
surname VARCHAR(255) NULL,
phone VARCHAR(255) NULL,
email VARCHAR(255) NULL,
birth_date VARCHAR(255) NULL,
country VARCHAR(255) NULL,
city VARCHAR(255) NULL,
postal_code VARCHAR(255) NULL,
address VARCHAR(255) NULL,
signup_date VARCHAR(255) NULL,
user_segment VARCHAR(255) NULL,
income_band VARCHAR(255) NULL,
region VARCHAR(255) NULL
);
-- Dimention table companies:
CREATE TABLE IF NOT EXISTS companies (
company_id VARCHAR(255) NULL,
company_name VARCHAR(255) NULL,
phone VARCHAR(255) NULL,
email VARCHAR(255) NULL,
country VARCHAR(255) NULL,
website VARCHAR(255) NULL,
merchant_category VARCHAR(255) NULL,
merchant_price_position VARCHAR(255) NULL
);

    -- -- Dimention table credit_cards
CREATE TABLE IF NOT EXISTS credit_cards (
id VARCHAR(255) NULL,
user_id VARCHAR(255) NULL,
iban VARCHAR(255) NULL,
pan VARCHAR(255) NULL,
pin VARCHAR(255) NULL,
cvv VARCHAR(255) NULL,
track1 VARCHAR(255) NULL,
track2 VARCHAR(255) NULL,
expiring_date VARCHAR(255) NULL,
card_type VARCHAR(255) NULL,
```

```sql
    card_renewal_flag VARCHAR(255) NULL
    );

    -- Table of facts transactions:
CREATE TABLE IF NOT EXISTS transactions (
    id VARCHAR(255) NULL,
    card_id VARCHAR(255) NULL,
    business_id VARCHAR(255) NULL,
    timestamp VARCHAR(255) NULL,
    amount VARCHAR(255) NULL,
    declined VARCHAR(255) NULL,
    product_ids VARCHAR(255) NULL,
    user_id VARCHAR(255) NULL,
    lat VARCHAR(255) NULL,
    longitude VARCHAR(255) NULL,
    discount_amount VARCHAR(255) NULL,
    tax_amount VARCHAR(255) NULL,
    shipping_amount VARCHAR(255) NULL,
    channel VARCHAR(255) NULL,
    campaign_id VARCHAR(255) NULL,
    device_type VARCHAR(255) NULL,
    is_international VARCHAR(255) NULL,
    decline_reason VARCHAR(255) NULL,
    distance_km VARCHAR(255) NULL
    );

        -- Load the data to USERS:
    LOAD DATA
    INFILE "C:/Users/sarab/Downloads/needed/ex8/N1-Ex.8__american_users.csv"
    INTO TABLE users
    FIELDS TERMINATED BY ','
    ENCLOSED BY '"'
    IGNORE 1 ROWS
    (id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band)
    SET region = 'America';

    LOAD DATA
    INFILE "C:/Users/sarab/Downloads/needed/ex8/N1-Ex.8__european_users.csv"
    INTO TABLE users
    FIELDS TERMINATED BY ','
    ENCLOSED BY '"'
    IGNORE 1 ROWS
    (id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band)
    SET region = 'Europe';

    -- birth_date updated to the DATE sql form:
    UPDATE users
    SET birth_date = STR_TO_DATE(birth_date, '%b %e, %Y')
    LIMIT 5000;

    -- Real types of columns and PK:
    ALTER TABLE users
    MODIFY id INT PRIMARY KEY,
    MODIFY name VARCHAR(255) NOT NULL,
    MODIFY surname VARCHAR(255) NOT NULL,
    MODIFY birth_date DATE,
    MODIFY signup_date DATE;

    -- Load the data to COMPANIES:
    LOAD DATA
    INFILE "C:/Users/sarab/Downloads/needed/ex8/N1-Ex.8__companies.csv"
    INTO TABLE companies
    FIELDS TERMINATED BY ','
    ENCLOSED BY '"'
    IGNORE 1 ROWS
    (company_id, company_name, phone, email, country, website, merchant_category, merchant_price_position);

    --  Real types of columns and PK:
    ALTER TABLE companies
    MODIFY company_id VARCHAR(20) PRIMARY KEY,
```

```sql
    MODIFY company_name VARCHAR(255) NOT NULL;

    -- Load the data to CREDIT_CARDS
    LOAD DATA
    INFILE "C:/Users/sarab/Downloads/needed/ex8/N1-Ex.8__credit_cards.csv"
    INTO TABLE credit_cards
    FIELDS TERMINATED BY ','
    ENCLOSED BY '"'
    IGNORE 1 ROWS
    (id, user_id, iban, pan, pin, cvv, track1, track2, expiring_date, card_type, card_renewal_flag);

    -- expiring_date updated to the DATE sql form:
    UPDATE credit_cards
    SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y')
    LIMIT 5000;

    -- Real types of columns and PK:
    ALTER TABLE credit_cards
    MODIFY id VARCHAR(20) PRIMARY KEY,
    MODIFY user_id INT,
    MODIFY expiring_date DATE,
    MODIFY card_renewal_flag TINYINT;

    -- The seperator here is ";"
    LOAD DATA
    INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8_transactions.csv'
    INTO TABLE transactions
    FIELDS TERMINATED BY ';'
    ENCLOSED BY '"'
    IGNORE 1 ROWS
    (id, card_id, business_id, timestamp, amount, declined, product_ids, user_id, lat, longitude, discount_amount, tax_amount, shipping_amount,
      channel, campaign_id, device_type, is_inernational, decline_reason, distance_km);

    -- Tipamos la tabla de hechos
    ALTER TABLE transactions
    MODIFY id VARCHAR(36) PRIMARY KEY,
    MODIFY card_id VARCHAR(10),
    MODIFY business_id VARCHAR(10),
    MODIFY timestamp DATETIME NOT NULL,
    MODIFY amount DECIMAL(10,2) NOT NULL,
    MODIFY declined TINYINT,
    MODIFY user_id INT,
    MODIFY lat DOUBLE,
    MODIFY longitude DOUBLE,
    MODIFY discount_amount DECIMAL(10,2),
    MODIFY tax_amount DECIMAL(10,2),
    MODIFY shipping_amount DECIMAL(10,2),
    MODIFY is_international TINYINT,
    MODIFY distance_km DECIMAL(10,2);

    -- Conecta la tabla de hechos con la dimension de tarjetas.
    ALTER TABLE transactions
    ADD CONSTRAINT transaction_credit_card_fk
    FOREIGN KEY (card_id)
    REFERENCES credit_cards(id)
    ON DELETE SET NULL;

    -- Conecta la tabla de hechos con la dimension de empresas.
    ALTER TABLE transactions
    ADD CONSTRAINT transaction_company_fk
    FOREIGN KEY (business_id)
    REFERENCES companies(company_id)
    ON DELETE SET NULL;

    -- Conecta la tabla de hechos con la dimension de usuarios.
    ALTER TABLE transactions
    ADD CONSTRAINT transaction_user_fk
    FOREIGN KEY (user_id)
    REFERENCES users(id)
    ON DELETE SET NULL;
```

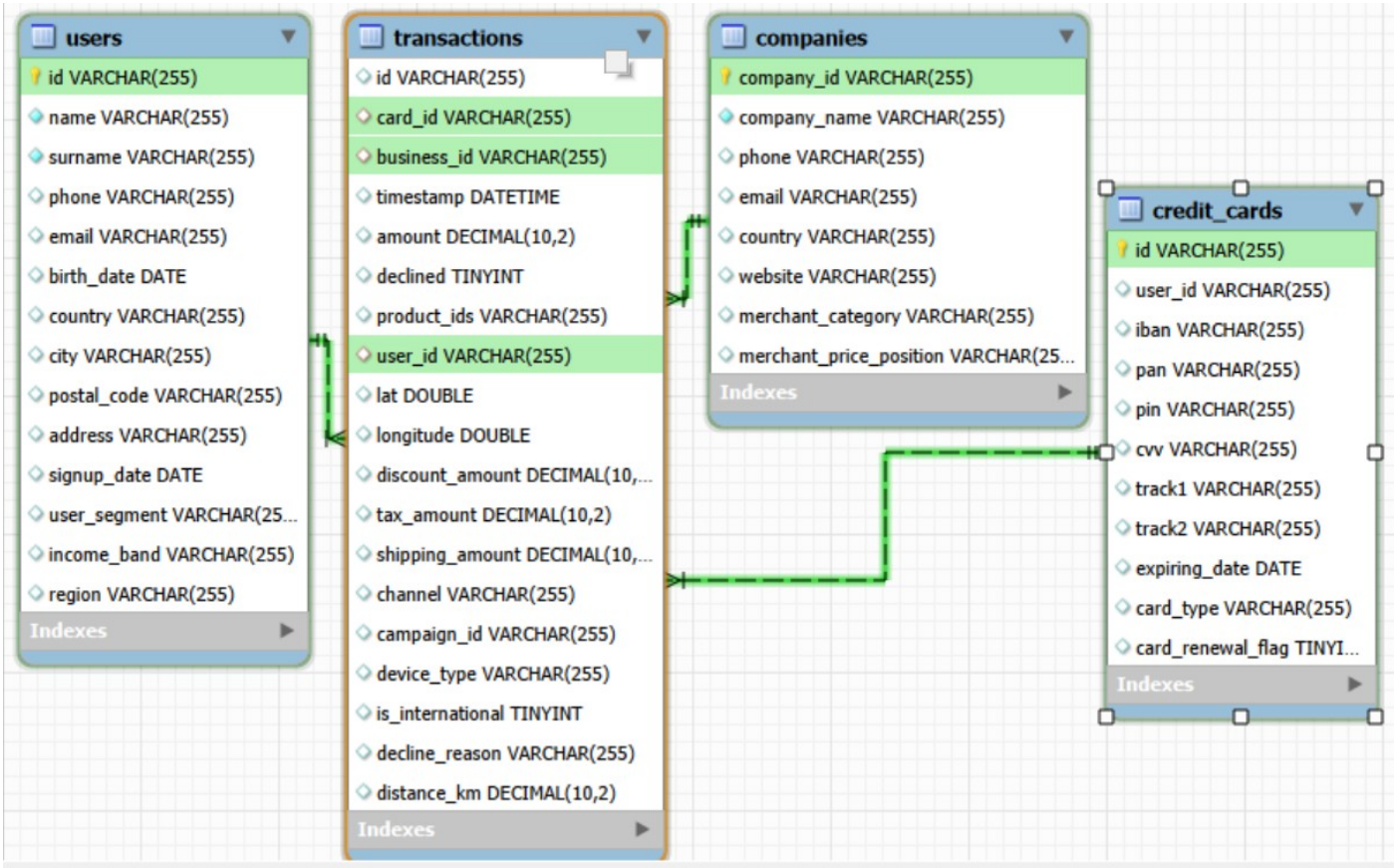
Proof of execution:

Output

Action Output

#	Time	Action	Message	Duration / Fetch
304	19:39:46	CREATE DATABASE IF NOT EXISTS sales	1 row(s) affected	0.000 sec
305	19:39:46	USE sales	0 row(s) affected	0.000 sec
306	19:39:46	SET GLOBAL local_infile = 1	0 row(s) affected	0.000 sec
307	19:39:46	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
308	19:39:46	CREATE TABLE users (id VARCHAR(255) NULL, name VARCHAR(255) NUL...	0 row(s) affected	0.016 sec
309	19:39:46	CREATE TABLE companies (company_id VARCHAR(255) NULL, company_n...	0 row(s) affected	0.031 sec
310	19:39:46	CREATE TABLE credit_cards (id VARCHAR(255) NULL, user_id VARCHAR(2...	0 row(s) affected	0.016 sec
311	19:39:46	CREATE TABLE transactions (id VARCHAR(255) NULL, card_id VARCHAR(2...	0 row(s) affected	0.047 sec
312	19:39:46	LOAD DATA LOCAL INFILE "C:/Users/sarab/Downloads/needed/ex8/N1-Ex.8__a...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.031 sec
313	19:39:46	LOAD DATA LOCAL INFILE "C:/Users/sarab/Downloads/needed/ex8/N1-Ex.8__e...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.109 sec
314	19:39:46	UPDATE users SET birth_date = DATE_FORMAT(STR_TO_DATE(birth_date, "%...	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0	0.188 sec
315	19:39:46	UPDATE users SET signup_date = DATE_FORMAT(STR_TO_DATE(signup_dat...	0 row(s) affected Rows matched: 0 Changed: 0 Warnings: 0	0.015 sec
316	19:39:47	ALTER TABLE users MODIFY id VARCHAR(255) NOT NULL, MODIFY name VARC...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.203 sec
317	19:39:47	LOAD DATA LOCAL INFILE "C:/Users/sarab/Downloads/needed/ex8/N1-Ex.8__c...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.000 sec
318	19:39:47	ALTER TABLE companies MODIFY company_id VARCHAR(255) NOT NULL, MODI...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.141 sec
319	19:39:47	LOAD DATA LOCAL INFILE "C:/Users/sarab/Downloads/needed/ex8/N1-Ex.8__cr...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.171 sec
320	19:39:47	UPDATE credit_cards SET expiring_date = DATE_FORMAT(STR_TO_DATE(ex...	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0	0.219 sec
321	19:39:47	ALTER TABLE credit_cards MODIFY id VARCHAR(255) NOT NULL, MODIFY user_...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.172 sec
322	19:39:47	LOAD DATA LOCAL INFILE "C:/Users/sarab/Downloads/needed/ex8/N1-Ex.8__tr...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0	2.500 sec
323	19:39:50	ALTER TABLE transactions MODIFY id VARCHAR(255), MODIFY card_id VARCHA...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	1.781 sec
324	19:39:52	ALTER TABLE transactions ADD CONSTRAINT transaction_credit_card_fk FOREI...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	2.375 sec
325	19:39:54	ALTER TABLE transactions ADD CONSTRAINT transaction_company_fk FOREIG...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	2.890 sec
326	19:39:57	ALTER TABLE transactions ADD CONSTRAINT transaction_user_fk FOREIGN KE...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	3.750 sec
327	19:40:01	SET SQL_SAFE_UPDATES = 1	0 row(s) affected	0.000 sec
328	19:40:01	SHOW TABLES	4 row(s) returned	0.000 sec / 0.000 sec
329	19:40:01	DESCRIBE users	14 row(s) returned	0.000 sec / 0.000 sec
330	19:40:01	DESCRIBE companies	8 row(s) returned	0.000 sec / 0.000 sec
331	19:40:01	DESCRIBE credit_cards	11 row(s) returned	0.000 sec / 0.000 sec
332	19:40:01	DESCRIBE transactions	19 row(s) returned	0.016 sec / 0.000 sec

Diagram:



Ex.9

```
435             -- Ex.9
436
437         -- Perform a subquery that shows all users with more than 80 transactions using at least 2 tables.
438
439 •    SELECT u.id, u.name, u.surname
440      FROM users AS u
441      WHERE u.id in(
442          SELECT t.user_id
443          FROM transactions AS t
444          GROUP BY t.user_id
445          HAVING COUNT(*) > 80);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	name	surname
185	Molly	Gilliam
289	Dxwgi	Hwcru
318	Bnyr	Astuw
454	Sfzzoh	Xgvfridxs

users 26 × Read Or

Output

Action Output

#	Time	Action	Message	Duration / Fetch
332	19:40:01	DESCRIBE transactions	19 row(s) returned	0.016 sec / 0.000 sec
333	19:47:05	SELECT u.id, u.name, u.surname FROM users AS u WHERE u.id in(SELECT t.user...	4 row(s) returned	0.157 sec / 0.000 sec

Ex.10

```
449         -- Shows the average amount per IBAN of credit cards in the company Donec Ltd, uses at least 2 tables.
450         -- company donec: iban: avg amount of credit cards:
451
452 •    SELECT cc.iban, ROUND(AVG(t.amount), 2) AS avg_amount
453      FROM credit_cards AS cc
454      INNER JOIN transactions AS t
455          ON cc.id = t.card_id
456      INNER JOIN companies AS cp
457          ON t.business_id = cp.company_id
458      WHERE cp.company_name = "Donec Ltd"
459      GROUP BY cc.iban;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

iban	avg_amount
US32110913566634207968934	144.44
CA76127409248451543987632	131.29
CA36791090618936446838870	217.26
CA53681144633113119329396	534.55
US28676381412179544810577	395.22

Result 27 × Read Or

Output

Action Output

#	Time	Action	Message	Duration / Fetch
333	19:47:05	SELECT u.id, u.name, u.surname FROM users AS u WHERE u.id in(SELECT t.user...	4 row(s) returned	0.157 sec / 0.000 sec
334	19:51:01	SELECT cc.iban, ROUND(AVG(t.amount), 2) AS avg_amount FROM credit_cards A...	371 row(s) returned	0.016 sec / 0.000 sec

Nivel 2

Ex.1

Identifies the five days that generated the highest amount of revenue for your business from sales. It shows the date of each transaction along with the total sales.

```
462         -- Ex.1
463
464     -- Identifies the five days that generated the highest amount of revenue for your business from sales. It shows the date of each
        transaction along with the total sales.
465 •   SELECT
466         DATE(`timestamp`) AS transaction_date,
467         SUM(amount) AS total_sales
468     FROM transactions
469     GROUP BY DATE(`timestamp`)
470     ORDER BY total_sales DESC
471     LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

transaction_date	total_sales
2019-11-18	15840.24
2023-02-20	15728.40
2022-12-13	15129.20
2019-03-18	14896.47
2017-12-20	14367.96

Result 28 ×

Output

Action Output

#	Time	Action	Message	Duration / Fetch
334	19:51:01	SELECT cc.iban, ROUND(AVG(t.amount), 2) AS avg_amount FROM credit_cards A...	371 row(s) returned	0.016 sec / 0.000 sec
335	19:58:23	SELECT DATE(`timestamp`) AS transaction_date, SUM(amount) AS total_sales...	5 row(s) returned	0.297 sec / 0.000 sec

Ex.2

It presents the name, telephone number, country, date and amount of those companies that carried out transactions with a value between 350 and 400 euros and on one of these dates: April 29, 2015, July 20, 2018 and March 13, 2024. Sort the results from highest to lowest amount.

```
473             -- Ex.2
474 •   SELECT companies.company_name,
475     companies.phone,
476     companies.country,
477     DATE(transactions. timestamp) AS date,
478     transactions.amount
479     FROM companies
480     INNER JOIN transactions ON transactions.business_id = companies.company_id
481     WHERE transactions.amount BETWEEN 350 AND 400
482     AND DATE(transactions.timestamp ) IN ('2015-04-29', '2018-07-20', '2024-03-13')
483     ORDER BY transactions.amount DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	phone	country	date	amount
Fringilla LLC	+1 150 784 1716	New Zealand	2015-04-29	390.33
Auctor Mauris Vel LLP	+32 133 447 8239	United States	2024-03-13	383.58
Mauris Institute	+75 473 619 9472	Sweden	2015-04-29	369.20
Pede Cum Ltd	+26 625 798 6126	Norway	2018-07-20	358.64
Netus Et Malesuada Ltd	+52 818 402 1638	Netherlands	2024-03-13	352.21

Result 29 ×

Output

Action Output

#	Time	Action	Message	Duration / Fetch
335	19:58:23	SELECT DATE(`timestamp`) AS transaction_date, SUM(amount) AS total_sales...	5 row(s) returned	0.297 sec / 0.000 sec
336	20:02:08	SELECT companies.company_name, companies.phone, companies.country, DATE(t...	6 row(s) returned	0.187 sec / 0.000 sec

Ex.3

We need to optimize the allocation of resources and it will depend on the operational capacity required, so they ask you for information on the number of transactions that companies carry out, but the human resources department is demanding and wants a list of companies where you specify whether they have equal to or more than 400 transactions or less.

```
487             -- Ex.3
488     -- Company Transactions by Date and Amount
489 •   SELECT cp.company_name, COUNT(*) AS total_transactions,
490         CASE
491             WHEN COUNT(*) >= 400 THEN '400 or more' ELSE 'Less than 400'
492         END AS transaction_category
493     FROM transactions AS t
494     INNER JOIN companies AS cp
495         ON t.business_id = cp.company_id
496     GROUP BY cp.company_id, cp.company_name
497     ORDER BY transaction_category DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	total_transactions	transaction_category
Nec Luctus LLC	399	Less than 400
Fringilla LLC	397	Less than 400
Lorem Eu Incorporated	380	Less than 400
Ac Fermentum Incorporated	2401	400 or more
Magna A Neque Industries	410	400 or more

Result 30 ×

Output

Action Output

#	Time	Action	Message	Duration / Fetch
336	20:02:08	SELECT companies.company_name, companies.phone, companies.country, DATE(t...	6 row(s) returned	0.187 sec / 0.000 sec
337	20:06:59	SELECT cp.company_name, COUNT(*) AS total_transactions, CASE WHEN C...	100 row(s) returned	0.407 sec / 0.000 sec

Ex.4

Remove from the transaction table the record with ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD from the database.

```
501
502         -- Exercise 4
503     -- Delete transaction:
504 •   SELECT * FROM transactions
505     WHERE id LIKE "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";
506
507 •   SET SQL_SAFE_UPDATES = 0;
508 •   DELETE FROM transactions
509     WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
510 •   SET SQL_SAFE_UPDATES = 1;
511
512 •   SELECT * FROM transactions
513     WHERE id LIKE "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";
514
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
340	20:11:09	SELECT * FROM transactions WHERE id LIKE "000447FE-B650-4DCF-85DE-C7ED...	1 row(s) returned	0.281 sec / 0.000 sec
341	20:11:09	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
342	20:11:09	DELETE FROM transactions WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1...	1 row(s) affected	0.421 sec
343	20:11:10	SET SQL_SAFE_UPDATES = 1	0 row(s) affected	0.000 sec
344	20:11:10	SELECT * FROM transactions WHERE id LIKE "000447FE-B650-4DCF-85DE-C7ED...	0 row(s) returned	0.282 sec / 0.000 sec

Ex.5

The marketing department wants to have access to specific information to perform effective analysis and strategies. It has been requested to create a view that provides key details about the companies and their transactions. You will need to create a view called VistaMarketing that contains the following information: Company name. Contact phone number. Country of residence. Average purchase made by each company. Present the view created, ordering the data from highest to lowest average purchase.

```
518     -- VistaMarketing
519 •   CREATE OR REPLACE VIEW VistaMarketing AS
520     SELECT cp.company_name, cp.phone, cp.country, ROUND(AVG(amount), 2) AS avg_purchase
521     FROM transactions AS t
522     INNER JOIN companies AS cp ON t.business_id = cp.company_id
523     GROUP BY cp.company_id, cp.company_name, cp.phone, cp.country
524     ORDER BY avg_purchase DESC;
525
526 •   SELECT *
527     FROM VistaMarketing
528     ORDER BY avg_purchase DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	phone	country	avg_purchase
Ac Fermentum Incorporated	+64 601 244 6320	Germany	330.77
Ut Semper Foundation	+10 283 500 6449	Sweden	307.37
Orci Adipiscing Limited	+60 897 390 5820	United Kingdom	306.27
Dolor Vitae Limited	+6 421 567 9195	France	305.95
Nunc In Foundation	+13 267 741 2619	Italy	305.40

VistaMarketing 34 ×

Output

Action Output

#	Time	Action	Message	Duration / Fetch
346	20:18:56	CREATE OR REPLACE VIEW VistaMarketing AS SELECT cp.company_name, cp.p...	0 row(s) affected	0.047 sec
347	20:18:56	SELECT * FROM VistaMarketing ORDER BY avg_purchase DESC	100 row(s) returned	1.047 sec / 0.000 sec

Nivel 3

Ex.1

Create a new table that reflects the status of credit cards based on whether the last three transactions have been declined then it is inactive, if at least one is not declined then it is active. Based on this table answer:

👉*How many cards are active?*

```sql
-- -- Nivel 3
-- Ex.1
-- Select the three most recent transactions for each card

-- Create the status table
CREATE TABLE credit_card_status (
    credit_card VARCHAR(255),
    transactions TEXT,
    status VARCHAR(20)
);

-- Insert the three most recent transactions per card
INSERT INTO credit_card_status (
    credit_card,
    transactions,
    status
)
WITH ranked_transactions AS (
    SELECT
        t.card_id,
        t.timestamp,
        t.declined,
        ROW_NUMBER() OVER (
           PARTITION BY t.card_id
           ORDER BY t.timestamp DESC, t.id DESC
        ) AS row_num
    FROM transactions AS t
    WHERE t.card_id IS NOT NULL
)

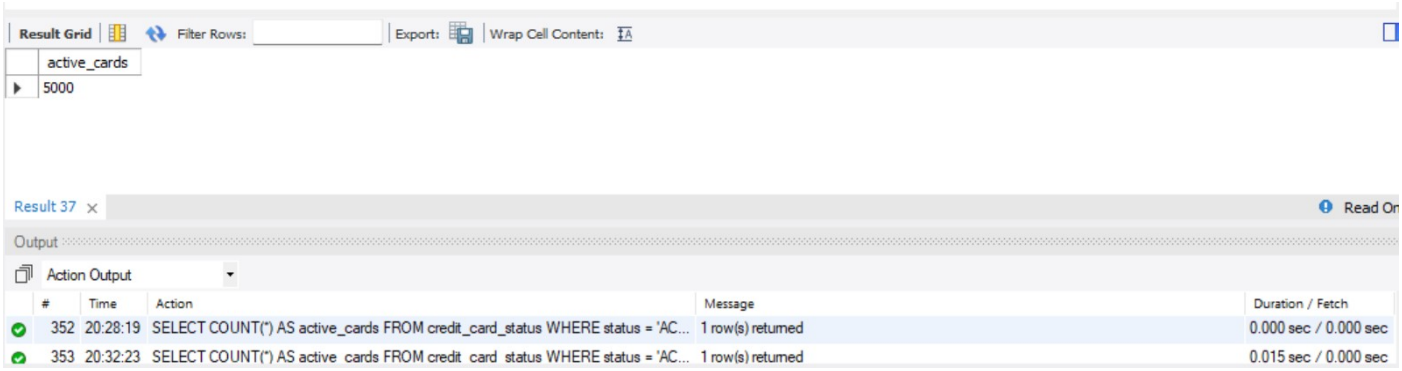
SELECT
    rt.card_id,

    GROUP_CONCAT(
        rt.declined
        ORDER BY rt.row_num
        SEPARATOR ', '
    ) AS transactions,

    CASE
        WHEN COUNT(*) = 3
             AND SUM(
                 CASE
                     WHEN rt.declined = 1 THEN 1
                     ELSE 0
                 END
             ) = 3
        THEN 'INACTIVE'
        ELSE 'ACTIVE'
    END AS status

FROM ranked_transactions AS rt
WHERE rt.row_num <= 3
GROUP BY rt.card_id;


-- Count active credit cards
SELECT COUNT(*) AS active_cards
FROM credit_card_status
WHERE status = 'ACTIVE';
```



- All 5000 cards are “active”. There are no “inactive” cards.

Ex.2

Create a table with which we can join the data from the products.csv file with the database created (since until now we could not do it), taking into account that from transaction you have product_ids. Generate the following query:

👉*We need to know the number of times each product has been sold.*

```
    -- Ex.2
DROP table products;
-- Create a dimention table products:

CREATE TABLE products (
    id VARCHAR(255),
    product_name VARCHAR(255),
    price VARCHAR(255),
    colour VARCHAR(255),
    weight VARCHAR(255),
    warehouse_id VARCHAR(255),
    category VARCHAR(255),
    brand VARCHAR(255),
    cost VARCHAR(255),
    launch_date VARCHAR(255)
);


-- 2. LOAD PRODUCTS DATA

LOAD DATA LOCAL INFILE
'C:/Users/sarab/Downloads/needed/N1-Ex.8__products.csv'
INTO TABLE products
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
IGNORE 1 ROWS
(id, product_name, price, colour, weight, warehouse_id,
 category, brand, cost, launch_date);


-- 3. CLEAN PRICE AND COST

SET SQL_SAFE_UPDATES = 0;
UPDATE products
SET
    price = REPLACE(TRIM(price), '$', ''),
    cost = REPLACE(TRIM(cost), '$', '')
WHERE price IS NOT NULL
   OR cost IS NOT NULL;
SET SQL_SAFE_UPDATES = 1;

-- 4. MODIFY COLUMN TYPES AND CREATE PRIMARY KEY


ALTER TABLE products
MODIFY id INT NOT NULL,
```

```
MODIFY weight DECIMAL(10,2),
MODIFY cost DECIMAL(10,2),
MODIFY launch_date DATE,
ADD PRIMARY KEY (id);


-- 5. MAKE SURE TRANSACTIONS.ID IS A PRIMARY KEY


-- Run this only if transactions.id does not already
-- have a PRIMARY KEY.

ALTER TABLE transactions
ADD PRIMARY KEY (id);


-- 6. CREATE PRODUCT_TRANSACTION TABLE


CREATE TABLE product_transaction (
    transaction_id VARCHAR(255) NOT NULL,
    product_id INT NOT NULL,

    PRIMARY KEY (transaction_id, product_id),

    CONSTRAINT product_transaction_transaction_fk
        FOREIGN KEY (transaction_id)
        REFERENCES transactions(id),

    CONSTRAINT product_transaction_product_fk
        FOREIGN KEY (product_id)
        REFERENCES products(id)
);


-- 7. SPLIT PRODUCT_IDS INTO INDIVIDUAL ROWS


INSERT INTO product_transaction (
    transaction_id,
    product_id
)
WITH RECURSIVE product_transactions AS (

    -- First product ID from each transaction
    SELECT
        t.id AS transaction_id,

        TRIM(
            SUBSTRING_INDEX(t.product_ids, ',', 1)
        ) AS product_id,

        CASE
            WHEN LOCATE(',', t.product_ids) > 0
            THEN TRIM(
                SUBSTRING(
                    t.product_ids,
                    LOCATE(',', t.product_ids) + 1
                )
            )
            ELSE NULL
        END AS remaining_ids

    FROM transactions AS t
    WHERE t.product_ids IS NOT NULL
      AND TRIM(t.product_ids) <> ''

    UNION ALL
```

```sql
    -- Extract the next product ID
    SELECT
        pt.transaction_id,

        TRIM(
            SUBSTRING_INDEX(pt.remaining_ids, ',', 1)
        ) AS product_id,

        CASE
            WHEN LOCATE(',', pt.remaining_ids) > 0
            THEN TRIM(
                SUBSTRING(
                    pt.remaining_ids,
                    LOCATE(',', pt.remaining_ids) + 1
                )
            )
            ELSE NULL
        END AS remaining_ids

    FROM product_transactions AS pt
    WHERE pt.remaining_ids IS NOT NULL
      AND pt.remaining_ids <> ''
)
SELECT
    transaction_id,
    CAST(product_id AS UNSIGNED) AS product_id
FROM product_transactions
WHERE product_id <> '';



-- 8. CHECK RESULTS


SELECT * FROM products;

SELECT * FROM product_transaction;

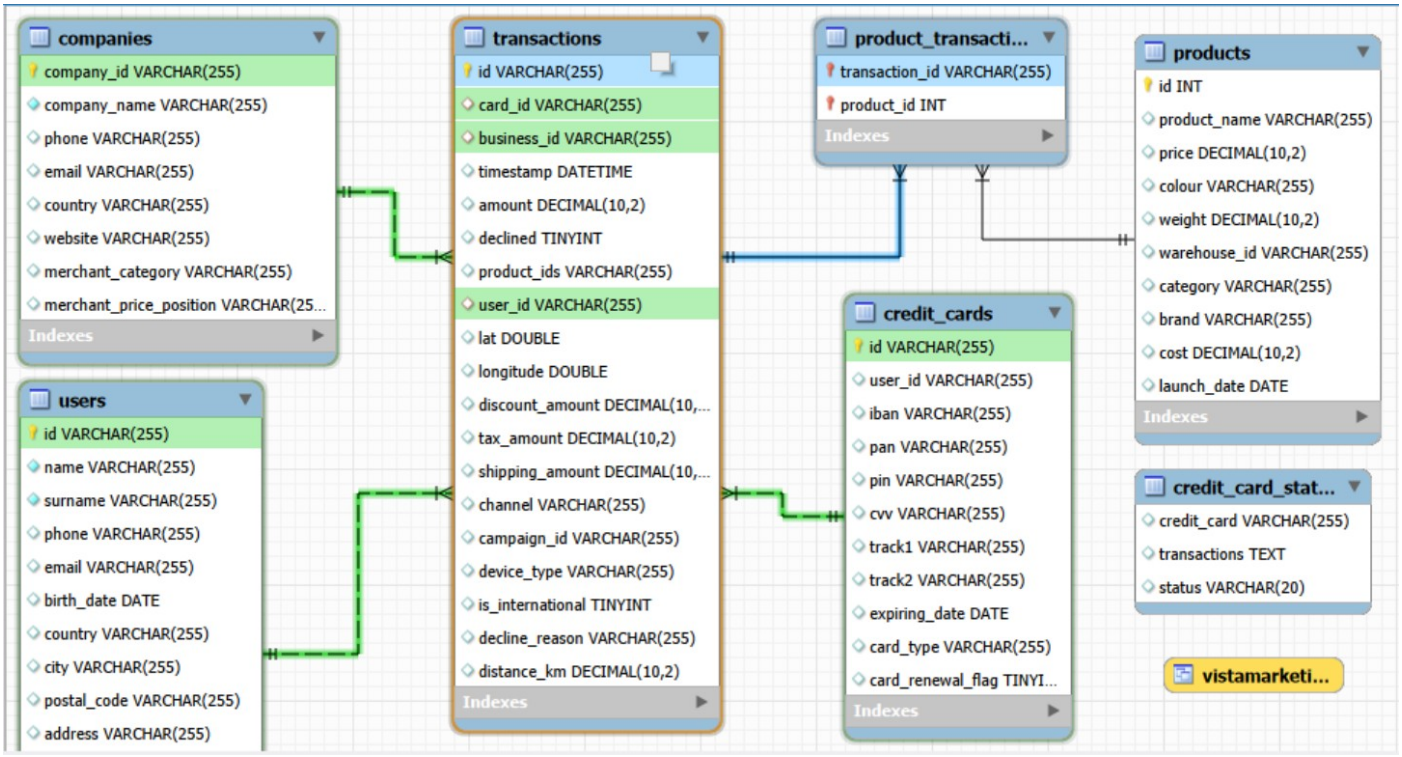
SELECT COUNT(*) AS total_product_transactions
FROM product_transaction;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
388	21:29:37	CREATE TABLE products (id VARCHAR(255), product_name VARCHAR(255),...	0 row(s) affected	0.078 sec
389	21:29:37	LOAD DATA LOCAL INFILE 'C:/Users/sarab/Downloads/needed/N1-Ex.8__produc...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.016 sec
390	21:29:37	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
391	21:29:37	UPDATE products SET price = REPLACE(TRIM(price), '$', ''), cost = REPLACE...	100 row(s) affected Rows matched: 100 Changed: 100 Warnings: 0	0.015 sec
392	21:29:37	SET SQL_SAFE_UPDATES = 1	0 row(s) affected	0.000 sec
393	21:29:37	ALTER TABLE products MODIFY id INT NOT NULL, MODIFY price DECIMAL(10,2)...	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0	0.094 sec
394	21:29:37	ALTER TABLE transactions ADD PRIMARY KEY (id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	4.000 sec
395	21:29:41	CREATE TABLE product_transaction (transaction_id VARCHAR(255) NOT NULL...	0 row(s) affected	0.047 sec
396	21:29:41	INSERT INTO product_transaction (transaction_id, product_id) WITH RECUR...	253388 row(s) affected Records: 253388 Duplicates: 0 Warnings: 0	13.375 sec
397	21:29:55	SELECT * FROM products	100 row(s) returned	0.000 sec / 0.000 sec
398	21:29:55	SELECT * FROM product_transaction	253388 row(s) returned	0.000 sec / 0.265 sec
399	21:29:56	SELECT COUNT(*) AS total_product_transactions FROM product_transaction	1 row(s) returned	0.094 sec / 0.000 sec

Diagram:


companies
company_id VARCHAR(255)
company_name VARCHAR(255)
phone VARCHAR(255)
email VARCHAR(255)
country VARCHAR(255)
website VARCHAR(255)
merchant_category VARCHAR(255)
merchant_price_position VARCHAR(25...
Indexes
users
id VARCHAR(255)
name VARCHAR(255)
surname VARCHAR(255)
phone VARCHAR(255)
email VARCHAR(255)
birth_date DATE
country VARCHAR(255)
city VARCHAR(255)
postal_code VARCHAR(255)
address VARCHAR(255)
transactions
id VARCHAR(255)
card_id VARCHAR(255)
business_id VARCHAR(255)
timestamp DATETIME
amount DECIMAL(10,2)
declined TINYINT
product_ids VARCHAR(255)
user_id VARCHAR(255)
lat DOUBLE
longitude DOUBLE
discount_amount DECIMAL(10,...
tax_amount DECIMAL(10,2)
shipping_amount DECIMAL(10,...
channel VARCHAR(255)
campaign_id VARCHAR(255)
device_type VARCHAR(255)
is_international TINYINT
decline_reason VARCHAR(255)
distance_km DECIMAL(10,2)
Indexes
product_transacti...
transaction_id VARCHAR(255)
product_id INT
Indexes
credit_cards
id VARCHAR(255)
user_id VARCHAR(255)
iban VARCHAR(255)
pan VARCHAR(255)
pin VARCHAR(255)
cvv VARCHAR(255)
track1 VARCHAR(255)
track2 VARCHAR(255)
expiring_date DATE
card_type VARCHAR(255)
card_renewal_flag TINYI...
Indexes
products
id INT
product_name VARCHAR(255)
price DECIMAL(10,2)
colour VARCHAR(255)
weight DECIMAL(10,2)
warehouse_id VARCHAR(255)
category VARCHAR(255)
brand VARCHAR(255)
cost DECIMAL(10,2)
launch_date DATE
Indexes
credit_card_stat...
credit_card VARCHAR(255)
transactions TEXT
status VARCHAR(20)
vistamarketi...